ByteFlow: A Comparative Platform for Classical and Quantum Max-Cut Solutions

Christiaan Gouws
4332981
4332981@myuwc.ac.za

Noah Mphahlele
4323227
4323227@myuwc.ac.za

Refiloe Angela Mshiyo
4388044
4388044@myuwc.ac.za

Yolisa Mzwakali
4395423
4395423@myuwc.ac.za

Ntsekiseng Elliot Nyembe
4101764
4101764@myuwc.ac.za

Jude Joshua
4368679
4368679@myuwc.ac.za

ABSTRACT

ByteFlow is an interactive, web-based platform designed to bridge usability and educational gaps in quantum computing by enabling side-by-side benchmarking of classical and quantum optimisation algorithms on the Max-Cut problem. Synthesised from published literature, user surveys, and existing tool evaluations, ByteFlow addresses common barriers in quantum software workflows—such as steep learning curves, the absence of integrated classical baselines, and execution delays on cloud quantum hardware. Structured around a "virtual laboratory" mental model for students and researchers, the system focuses on smaller graphs—roughly single-digit to low double-digit node counts preferred for demonstration and learning scenarios—allowing users to explore problem instances without requiring deep underlying mathematical knowledge. Architecturally, ByteFlow integrates a classical solver module (executing standard Semidefinite Programming (SDP) algorithms) alongside a Quantum Approximate Optimisation Algorithm (QAOA) engine connected to IBM Quantum Cloud hardware via Qiskit Runtime sessions. To overcome remote queue timeouts and Application Programming Interface (API) authentication errors, the platform features an automated fallback handling module that transparently redirects execution to a local Qiskit Aer simulator with clear provenance logging. ByteFlow provides a comparative analytics dashboard that renders side-by-side metric visualisations—including graph partition layouts, wall-clock execution times, approximation ratios, and solution probabilities—delivering a practical, intuitive tool for evaluating quantum utility.

1 INTRODUCTION

Quantum computing is a growing area of computer science that offers new ways of solving difficult computational problems. One problem that can be used to explore this is the Max-Cut problem, where the goal is to divide the vertices of a graph into two groups while maximising the number of edges between the groups. Max-Cut can be solved using traditional classical algorithms, but quantum algorithms such as the Quantum Approximate Optimisation Algorithm provide another approach. However, learning and experimenting with quantum optimisation can be difficult for students and researchers. It often requires specialised knowledge, access to quantum hardware, and dealing with delays when using cloud-based quantum systems. Comparing classical and quantum results can also require several different tools. ByteFlow aims to address these challenges by providing an interactive web-based platform where users can create graphs, run classical and quantum algorithms, and compare their results in one easy-to-use environment.

2 USER MODEL

2.1 Data Collection

Due to time and access constraints, this research draws on secondary sources. The following findings are synthesised from published literature, official documentation, and community discussions. Below are the findings that shaped the User Model, Mental Model, and Requirements Analysis.

2.1.1 Documented Observations of Developers Using Existing Quantum Tools. Published tutorials and studies of developers using IBM Quantum Composer and Qiskit notebooks reveal a steep learning curve and persistent friction. The process of converting a graph description into a runnable circuit [13], such as an adjacency list into a Hamiltonian, involves multiple non-trivial steps that are a known barrier for novice developers.

A recurring behaviour documented in the literature is long queue times on shared quantum hardware [11]. This is reflected in recent research on hybrid systems, which emphasises the need for a fallback execution to classical surrogates when quantum resources are unavailable or slow [15].

A third theme is the demand for a classical baseline. The QAOA workflow is a hybrid process in which a classical optimiser finds optimal parameters, and the final solution requires domain knowledge to "sanity-check". Consequently, developers need to compare the quantum result with a classical alternative to understand its value, a concept central to measuring "quantum utility" [6].

2.1.2 Findings from Published Interviews and Surveys. Interview and survey studies of quantum computing students, developers, and educators identify three recurring needs. First, participants consistently report the absence of a systematic way to compare classical and quantum solutions on the same problem instance, and identify this as a gap in current educational tooling [6].

Second, larger graph instances are described as interesting but impractical for live or classroom use. Smaller graphs — roughly single-digit to low double-digit node counts — are preferred for demonstration and learning scenarios [9].

Third, educators and learners emphasise the need for a mode that completes reliably within a fixed time window, even when real hardware queues are long [11]. This concern is echoed in the broader QSE literature on hardware access variability and fallback execution.

Table 1: Review of existing quantum and classical interfaces

Tool	What worked well	What felt heavy for classroom use
1. IBM Quantum Experience / Composer [7]	Clear circuit visualiser; backend status	Long path from graph → Hamiltonian → circuit; no built-in classical baseline
2. Qiskit textbook notebooks [8]	Excellent explanations	Students must write a lot of boilerplate; comparison with classical is left as an exercise
3. Amazon Braket console [3]	Clean job history	Different SDK; not aligned with the tooling assumed by our project
4. Open-source Max-Cut visualisers (GitHub) [10]	Instant classical results and clean graph drawings	No quantum path at all
5. Online graph editors (e.g. graphonline.ru) [4]	Very fast graph input	No optimisation solvers

2.1.3 Existing-Tool Review. We reviewed publicly documented interfaces and tool workflows to identify interaction patterns relevant to our design. Table 1 summarises what each tool does well and where it falls short for classroom or demonstration use. The gap identified across all five tools is precisely the space our platform targets.

2.2 User Profiles

2.2.1 User/Researcher/Student.

- User role: End user
- Characteristics: Semi-proficient with technology, familiar with standard classical algorithms but not necessarily quantum computing. Will access and make use of software when needed for research or personal curiosity. Byteflow will be accessed with any electronic device that can access the internet with a sufficient connection. Users can be of any educational background and level.
- Primary needs/Goals: Access the "Byteflow" program and interface, view comparison of computing methods between standardized computing algorithms and quantum computing algorithms through various metrics and have an easy to use, intuitive user interface.

2.2.2 System administrator.

- User role: "Byteflow" system administrator
- Characteristics: Will monitor the systems activity daily, requires domain knowledge of user access to systems and allocation of cloud based computing resources on a network. Requires training in network administration and managing cloud computing resources.
- Primary needs/Goals: Monitor application users and computing resources to ensure optimal operations, exercising the rights to limit user access and computing resources.

2.2.3 IBM.

- User role: External system, Quantum computing resource supplier
- Characteristics: Access will be provided through Qiskit, constraints on computing resource limits are applied to operations using the resources.
- Primary needs/Goals: Serves as the provider of the IBM Quantum Cloud Computing resources for the Max Cut Problem; receives the algorithms and processes it on its servers, and returns the results to the users of the "Byteflow" interface.

2.3 UML Use Case Diagram

The use case diagram in Figure 1 models the use cases and actors involved. Unfortunately, the text in Figure 1 is small so we recommend zooming in using Ctrl + Scroll or Ctrl + Plus/Minus.

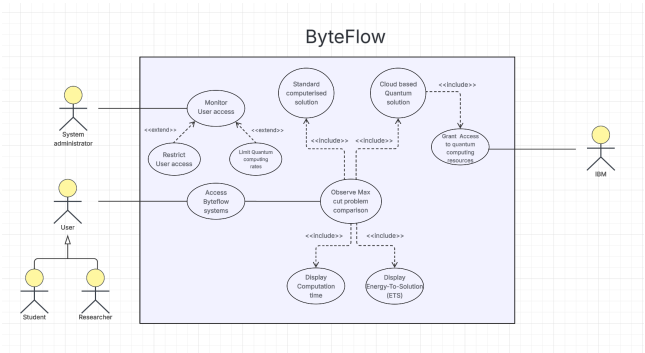


Figure 1: Use case diagram for the Max-Cut system.

3 MENTAL MODEL

3.1 System Perception

The user is expected to perceive the system as a tool for exploring or comparing different approaches to solving the same optimization problem. The application uses the Max Cut problem as an example. A Max Cut problem consists of nodes connected by edges and the objective is to divide the nodes into two groups while maximising the number or total weights of edges connecting two groups. The expected user interaction can be represented as Figure 2:

From the user's perspective, the technical processes occurring in the background should not need to be understood in order to use the application. For example, when the quantum approach is selected, the user does not need to understand the mathematical formulation of QAOA. The application will communicate the important information in an understandable form and present the resulting data visually.

The quantum component uses IBM Quantum services to provide access to quantum computing resources. From the user's perspective, IBM Quantum can therefore be viewed as a remote quantum resource used to perform the quantum part of the experiment. The application receives the resulting information and presents it to the user. The user's mental model can therefore be summarised as: Select a problem → Understand the problem → Run two approaches → Observe the results → Compare results → Learn from the comparison. This allows the application to introduce quantum computing through practical experimentation rather than requiring

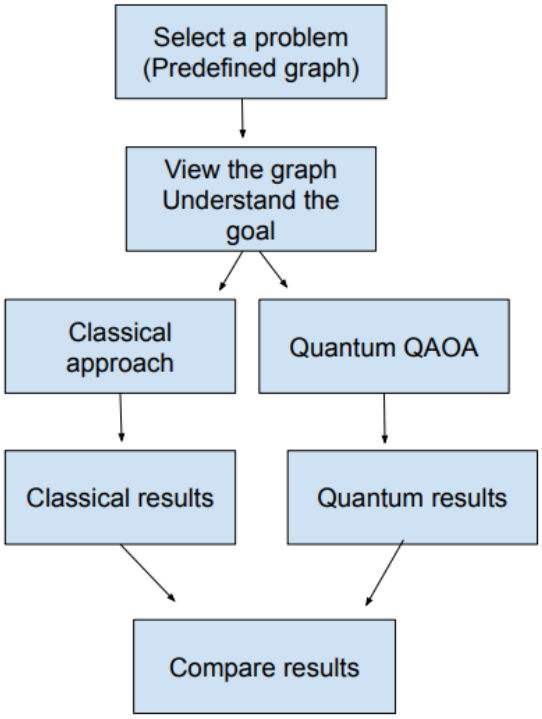


Figure 2: User Interaction diagram for the Max-Cut system.

users to understand the underlying technology before interacting with it.

3.2 Real-World Metaphors

The application can be compared to a virtual laboratory, where users can explore and investigate problems using different computational approaches. Similar to a physical lab, where different methods can be used to investigate the same problem, the application allows users to examine a predefined Max Cut problem using both classical and quantum computing.

Selecting a predefined Max Cut problem is similar to choosing an experiment, while viewing the graph represents examining the experimental setup. The classical solver represents a traditional computational method, whereas the quantum solver represents an alternative method of investigating the same problem. The results produced by both approaches act as observations that users can examine and compare.

This metaphor helps users understand the application as an interactive learning environment where they can explore optimization problems, observe the outcomes of different computational approaches and develop a better understanding of quantum computing without needing advanced knowledge of its underlying principles.

4 REQUIREMENTS ANALYSIS

4.1 Functional & Behavioral Requirements

4.1.1 Classical Solver Module (UC1).

- The system shall execute classical Max-Cut optimisation algorithms (not custom-coded) on the predefined graph [5].
- The system shall record the classical solution outputs which includes optimal cut values and wall-clock execution times [5].

4.1.2 Quantum Execution Module (UC2).

- The system shall compile and dispatch gate-based QAOA quantum circuits to IBM Quantum Cloud hardware using sessions [5, 16].
- The system shall display the execution status and the returned measurement shot distributions from IBM Quantum processing units [5, 16].

4.1.3 Fallback Handling Module (UC3).

- The system shall detect IBM execution timeouts, session expirations, or API errors and initiate fallback procedures to a local simulator.
- The system shall notify the user of the fallback and deliver simulated QAOA expectation values without interrupting the session [2].

4.1.4 Configuration & Security Module (UC4).

- The system shall provide secure and encrypted configuration and storage for user API tokens
- The system shall validate API token credentials and then verify QPU availability before dispatching.

4.1.5 Comparative Analytics Module (UC5).

- The system shall display comparative analytics, rendering side-by-side metrics for classical vs quantum solvers [2]. These include approximation ratios, wall-clock runtimes, etc [5, 16].

4.1.6 Scenario 1: User runs a classical Max-Cut.

(1) User selects classical solver (Goemans-Williamson SDP) for the graph → System highlights classical configuration options and enables the run button [5].
(2) User clicks the Run button → System dispatches the predefined matrix to the classical execution engine [5].
(3) System completes execution → System highlights the cut edges and displays the wall-clock execution time (metrics) [5].

4.1.7 Scenario 2: User runs a quantum Max-Cut with successful IBM execution.

(1) User selects the QAOA quantum solver, configures layer depth p, and inputs an API token → System validates the token and checks QPU backend status [5].
(2) User clicks the Run button → System constructs the QAOA circuit ansatz, transpiles it for the target IBM QPU, and initiates an IBM Runtime session [5, 16].
(3) IBM QPU returns measurement samples → System computes the quantum expectation value, extracts the highest-probability bitstring cut, and displays final metrics [5, 16].

4.1.8 Scenario 3: User runs a quantum Max-Cut that times out and falls back.

(1) User initiates QAOA execution (same as scenario 2) → System submits the job session and displays initial queue status.
(2) IBM Quantum Cloud backend exceeds the queue time-to-live threshold → System flags a remote execution timeout exception and triggers fallback handling to local Qiskit Aer Sim [2].
(3) Local sim completes → System renders simulated Max-Cut results clearly annotated with a local sim provenance label [2].

4.1.9 *Scenario 4: User views the comparatitive dashboard.*

(1) User navigates to the Comparative Analytics tab/dashboard after running both solvers → System fetches stored execution metrics [2].
(2) User selects specific classical and quantum run instances to benchmark → System renders dual metric visualisations displaying classical vs quantum cut partitions side-by-side [2].
(3) System compiles comparative metrics → System generates side-by-side charts comparing approximation ratios, solution probabilities, and execution runtimes [5, 16].

4.1.10 State Diagram. The state diagram in Figure 3 models the solver execution life cycle. Unfortunately, the text in Figure 3 is small so we recommend zooming in using Ctrl + Scroll or Ctrl + Plus/Minus.

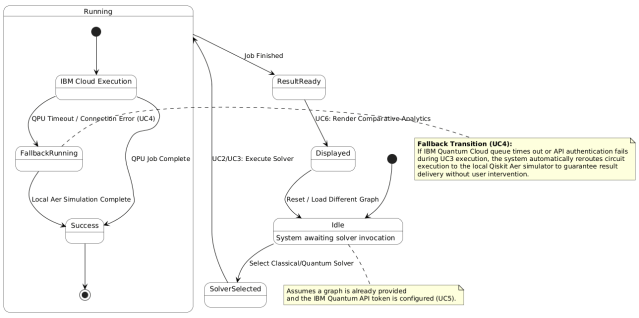


Figure 3: State diagram for the Max-Cut system.

4.2 Use Case Specifications

This section details the specific workflows of the system. The foundational process, UC1 (Execute Classical Max-Cut Solver), is first outlined in Table 2.

Table 2: Use Case UC1 — Execute Classical Max-Cut Solver

Field	Value
ID	UC1
Actor	User
Goal	Execute a classical optimisation algorithm on the loaded graph to compute baseline Max-Cut results.
Preconditions	A graph is loaded in the active workspace.
Trigger	User selects a classical algorithm and initiates execution.
Main Flow	1. System dispatches the graph adjacency matrix to the classical execution engine. 2. System computes the maximum cut partition, optimal cut value, and wall-clock runtime. 3. System displays the classical partition layout and stores solution metrics.
Alternate Flows	A1. Classical execution error → System displays an error message and prompts retry.
Postconditions	Classical Max-Cut metrics are saved for comparative analytics.
Related	None

The detailed specifications for the remaining use cases are provided in the following tables. Table 3 outlines the execution flow for UC2 (Quantum Max-Cut Solver), while Table 4 describes the automatic fallback procedures for UC3. The secure configuration of credentials is detailed in Table 5 (UC4), and the system's benchmarking capabilities are outlined in Table 6 (UC5).

Table 3: Use Case UC2 — Execute Quantum Max-Cut Solver (QAOA)

Field	Value
ID	UC2
Actor	User
Goal	Run a QAOA circuit on IBM Quantum Cloud hardware to obtain quantum Max-Cut results.
Preconditions	A graph is loaded and an IBM Quantum API token is configured (UC4).
Trigger	User configures QAOA parameters and initiates quantum execution.
Main Flow	1. System converts the graph to an Ising Spin Hamiltonian and builds the QAOA circuit ansatz. 2. System initiates an IBM Quantum Runtime session and submits variational execution jobs. 3. System streams parameter updates and renders QPU measurement shot distributions.
Alternate Flows	A1. IBM queue timeout or authentication failure → System triggers fallback to the local simulator per UC3.
Postconditions	Quantum Max-Cut results and parameter trajectories are stored for comparative analysis.
Related	≪include≫ UC4 (Configure IBM Quantum API Token)

Table 4: Use Case UC3 — Fallback to Local Simulator

Field	Value
ID	UC3
Actor	User
Goal	Automatically redirect QAOA execution to a local quantum simulator when remote cloud execution fails.
Preconditions	UC2 execution is active and has not yet returned a result.
Trigger	IBM Quantum Cloud queue timeout or authentication failure occurs during UC2.
Main Flow	1. System flags the remote execution failure and notifies the user of the simulation fallback. 2. System dispatches the QAOA circuit to the local Qiskit Aer simulator engine. 3. System calculates simulated expectation values and renders results with provenance labels.
Alternate Flows	A1. Simulator memory limit exceeded → System aborts the run and prompts the user to reduce graph size.
Postconditions	Simulated QAOA results are delivered without interrupting the user session.
Related	≪extend≫ UC2 (Execute Quantum Max-Cut Solver)

Table 5: Use Case UC4 — Configure IBM Quantum API Token

Field	Value
ID	UC4
Actor	User
Goal	Securely store and validate an IBM Quantum API token for remote QPU access.
Preconditions	User holds a valid IBM Quantum account credential.
Trigger	User accesses configuration settings, or UC2 detects a missing token.
Main Flow	1. User enters the IBM Quantum API token in the configuration field. 2. System validates the token and checks IBM Quantum backend availability. 3. System encrypts and stores the validated token for session use.
Alternate Flows	A1. Invalid API token → System displays an authentication error and requests valid credentials.
Postconditions	System is authenticated to dispatch execution jobs to IBM Quantum Cloud.
Related	None

Table 6: Use Case UC5 — View Comparative Analytics Dashboard

Field	Value
ID	UC5
Actor	User
Goal	Compare classical versus quantum Max-Cut results in a side-by-side visual dashboard.
Preconditions	At least one execution result exists (classical, quantum, or both).
Trigger	User navigates to the Comparative Analytics view.
Main Flow	1. System retrieves recorded execution artefacts for the active graph runs. 2. System renders side-by-side graph partitioning layouts and measurement distributions. 3. System plots comparative charts for approximation ratios, solution probabilities, and runtimes.
Alternate Flows	A1. Incomplete solver data → System displays placeholder guidance to execute the missing solver(s).
Postconditions	Comparative benchmarking visualisations and metrics are displayed.
Related	None

5 DESIGN MODEL

5.1 Data Design

The following section briefly explains the data design used in the prototype. The data design in the prototype is relational, linking user configurations, predefined problem instances, and execution artefacts for comparative benchmarking (traditional vs. quantum). The Entity-Relationship Diagram (ERD) in Figure 4 visualizes these concepts.

The diagram can be further explained with this list, which maps back to our user model:

- User & API_Token: Maps directly UC4 (Configuration & Security Module). These handle secure storage and validation of user credentials required to connect to the IBM Quantum Cloud.
- Graph_Instance stores predefined Max-Cut problem topologies selected in the virtual workspace.
- Execution_Run links users and graphs to specific algorithmic executions, tracking solver type (Classical SDP or QAOA) and parameters like layer depth p (UC1 & UC2).
- Execution_Result stores output metrics for the Comparative dashboard (UC5), including classical and quantum-specific data. This table stores both classical and quantum-specific metrics/data.

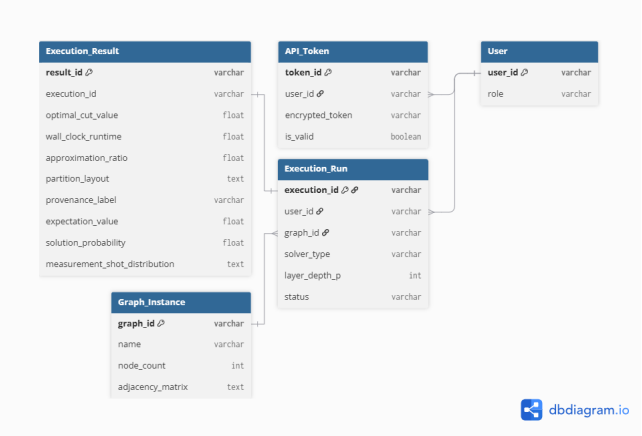


Figure 4: ERD Diagram for the Max-Cut system.

The provenance_label attribute in Execution_Result records whether a quantum result was executed on an IBM QPU or generated by the local Qiskit Aer simulator following a timeout.

5.2 Architectural Design

The prototype uses a multi-tiered client-server architecture, decoupling the frontend UI from classical and quantum solvers. It routes user inputs to execution engines while securely managing API credentials and job sessions. The System Architecture Diagram in Figure 5 highlights the main components in the prototype's architecture.

To further explain the architecture, here are a few points regarding the diagram:

- Web-Based Client serves as the users' workspace, containing graph information and the comparative dashboard. It communicates with the backend via HTTP/REST APIs to submit execution jobs and retrieve metrics.
- The Backend Server is the central controller, orchestrating system workflows and connecting to the database to store execution artefacts and user API tokens securely (UC4).
- The Classical Solver Module processes standard Semidefinite Programming (SDP) algorithms (UC1) [1, 12].
- The Quantum Engine & IBM Quantum Cloud constructs QAOA circuit ansatz and initiates Qiskit Runtime sessions with the IBM Quantum Cloud hardware (UC2).
- The Fallback Handling Module & Local Simulator handles faults and monitors the IBM Quantum Cloud connection, detecting remote queue timeouts or API authentication errors and redirecting execution to the internal Local Qiskit Aer Simulator (UC3).

5.3 Class Design

We can summarise and outline the design of the classes in this diagram. But more importantly, we can define each class and its uses in the following itemised list:

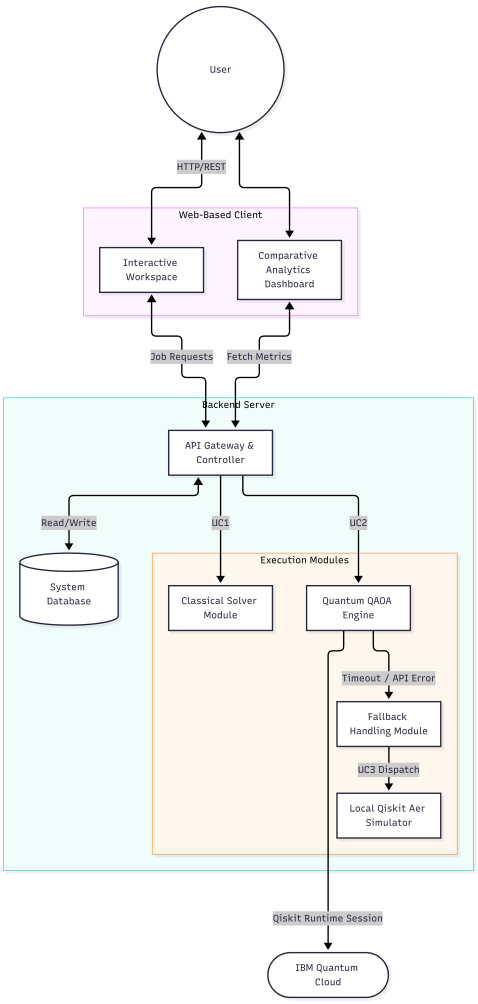


Figure 5: Architecture Diagram for the Max-Cut system.

- 'GraphWorkspace' manages the active graph instance, storing node data and adjacency matrices used by the path-finding solver.
- ClassicalSolver inherits graph data and runs the deterministic Semidefinite Programming algorithm, which is a method for finding approximate solutions to optimisation problems by relaxing them into a form that can be handled with matrix variables [1, 12].
- QiskitQAOASolver requires a layer depth parameter p, an integer controlling how many times the basic set of quantum gates is repeated in the circuit [14].
- FallbackManager tracks QiskitQAOASolver job status and reroutes the job if IBM queue time exceeds a set limit.

5.4 Interface Design

We use a RESTful API, so the Web-Based Client can send user requests and then display the results. Afterwards, the backend server processes the sent requests and manages computations, relying on the official Qiskit SDK to interact with external hardware.

- The internal API endpoints handle requests between the frontend and backend, allowing the client to trigger classical or quantum solves and receive results:
  - **POST /api/solve/classical:** The frontend sends the graph ID. The backend processes the data through ClassicalSolver and returns the partition layout and runtime.
  - **POST /api/solve/quantum:** The frontend sends the graph ID and layer depth p. This endpoint returns a job_id that the frontend polls for updates.
- After handling internal API endpoints, the system connects externally through the IBM interface:
  - The backend communicates with the IBM Quantum Cloud using the qiskit-ibm-runtime Python library. It establishes a WebSocket session to stream parameter updates back to the system and query queue status dynamically.

5.5 Component-Level Design

We'll examine the pseudocode of the Fallback Handling Module (UC3), which remains vital because it dynamically switches to a local simulator if the remote IBM hardware fails. With these sections done, we'll move on to the UI design and process.

Algorithm 1: Execute Quantum QAOA with Fallback Mechanism

```
function executeQuantumQAOA(graph, layer_depth, token):
    ▷ Step 1: Initialize external connection
    try
        session ← INITIALIZEIBMSESSION(token);
        ansatz ← BUILDQAOACIRCUIT(graph, layer_depth);
        ▷ Step 2: Submit job to IBM Quantum Cloud
        job ← session.RUN(ansatz);
        ▷ Step 3: Monitor queue status
        while job.status ≠ "COMPLETED" do
            if job.time_in_queue > TIMEOUT_LIMIT then
                throw TimeoutException(...);
            WAIT(5s);
        ▷ Step 4: Return remote results
        return PROCESSMETRICS(job.results(), "IBM QPU");
    catch TimeoutException or AuthenticationError
        ▷ Step 5: Fallback to Local Aer
        LOG("Remote failed. Initiating Fallback...");
        sim ← INITIALIZELOCALAERSIMULATOR;
        sim_job ← sim.RUN(ansatz);
        return
          PROCESSMETRICS(sim_job.results(), "Local Sim");
```

6 USER INTERFACE DESIGN & PROCESS

6.1 Interface Process

The design of the User Interface was created iteratively through four main steps:

6.1.1 ***User Analysis.*** We first started with a user analysis of various popular student learning and research-conducting software and what user interfaces they incorporate in their designs. Paying close attention to things like the colours used, how intuitive the user interface is to use and which one is best for conducting learning and knowledge retention. Heavy Boilerplate was one other factor of careful consideration during elicitation, finding a user interface that cut away the busywork and tedious setup that allowed users to quickly and accurately make use of the data comparison tools available.

6.1.2 ***Sketching.*** After we came up with a general idea of what design we wanted for the *Byteflow* learning program, we began our initial low-fidelity sketches of the rough view of the user interface and iteratively tweaked it with development feedback until we found a design we felt comfortable prototyping.

6.1.3 ***Prototyping.*** We then moved from a sketch into developing a wire-frame model based on the final iteration of the sketch, agreeing upon careful consideration of the layout, size of the various graphics and fonts and where we want the focal point of the screen to be for the user. The wire-frame helped solidify the final design we would then decide to roll out.

6.1.4 ***Evaluation.*** Finally, we conducted an evaluation phase where we gathered feedback from a sample of researchers and students to help solidify the design and fix any design faults, like fonts that were difficult to read or bad colours that were distracting. We then paid careful attention to issues such as difficulty in picking up the software design and understanding the user interface, wanting to ensure it was easy for students or researchers of varying technology literacy levels to pick up and understand. After considering and revising these factors, we arrived at a final design ready for implementation.

6.2 Screen Layouts

The screen layout for the User Interface is as follows:

6.2.1 ***Workspace.*** Consists of an overhead bar detailing the input for the comparison between traditional and quantum computing methods; underneath the bar, it is split into two windows, one for the traditional computing output and one for the quantum computing output, so the user can make an ample comparison of the data. The design utilises simple shapes and fonts and light grey tones, so it's easy on the user's eyes no matter what time of day the application is being used.

6.2.2 ***Graph Input Area.*** Provides the necessary tools for generating graphs and selecting datasets for the various comparisons. The user interface includes various buttons: one to reset the display of all information, to randomise the input for another observation to observe, and a button to switch the dual-channel display to only quantum computing or only traditional computing.

6.2.3 ***Configuration Panel.*** There will be user controls to adjust the layer depth p that resizes the windows as the user desires for the input, quantum computing and traditional computing windows. They will be able to save the configuration for when they use the application the next time.

6.2.4 ***Comparative Analytics Dashboard.*** There will be a side-by-side comparison of the various computation metrics between quantum computing and traditional computing using benchmarks of various performance metrics. The user will be able to adjust windows and switch to an individual display for each.

6.3 Usability Focus

To breakdown how the interface addresses issues identified in user research as follows:

6.3.1 ***Eliminating the Heavy Boilerplate.*** Standard quantum software workflows using the Qiskit SDK require writing repetitive, error-prone code to convert graph adjacency matrices into Ising Spin Hamiltonians, construct circuit ansatzes, configure transpiler passes, manage Qiskit Runtime sessions, and parse measurement shot arrays. ByteFlow automates this entire pipeline behind an interactive UI. Users configure experiments using visual controls (such as graph selection dropdowns, layer depth p inputs, and shot count controls), while the backend handles circuit generation, session management, and result processing transparently.

6.3.2 ***Reducing the Steep Learning Curve.*** ByteFlow uses a laboratory model to help users focus on the big ideas instead of getting stuck in technical details. It hides math, such as matrix exponentials or continuous semidefinite programming (SDP) relaxations. Instead, users work with visual graphs and can compare their quantum algorithm results side by side with classical SDP baselines. This means that even those without knowledge of quantum information theory can run experiments and understand how well their algorithms perform.

7 PROTOTYPE DEVELOPMENT

7.1 Prototyping Strategy

A throwaway prototyping approach was adopted using Figma to create a static visual representation of the proposed Virtual Max Cut Laboratory. The prototype was designed to demonstrate the main user interface, navigation and workflow involved in comparing classical and quantum approaches to the Max Cut optimisation problem.

The prototype consists of five main stages: Experiment, Connection, Execution, Fallback and Analytics. The experiment workspace screen allows users to select a predefined graph and configure the parameters for the classical and quantum solvers. The IBM Quantum Connection screen represents the setup required to connect to IBM Quantum Cloud. The Execution screen represents the solver execution process, while the Fallback screen illustrates the alternative approach when remote quantum execution cannot proceed. Finally, the Analytics screen is intended to present the results for comparison.

This approach allows the group to visualise the system's proposed functionality and interface before completing the actual implementation. It also provides an opportunity to identify possible usability issues and improve the design before further development.

7.2 Hardware Specifications

The proposed system will use both local computing and remote quantum computing. A laptop will be used to run the Qiskit Aer

simulator when local quantum circuit simulation is required. This will also support the proposed fallback mechanism when remote quantum execution cannot proceed. The simulator will execute locally using the laptop's available computing resources.

Remote quantum execution is intended to use IBM Quantum Cloud to access supported quantum processing unit (QPU) hardware. This will allow quantum circuits prepared using the Quantum Approximate Optimisation Algorithm (QAOA) to be submitted for execution on remote quantum hardware, subject to service availability and access requirements. The combination of local simulation and remote quantum execution is intended to provide an alternative when remote execution is unavailable or encounters problems.

7.3 Software & Tools

Figma was used to design the user interface and visual workflow of the Virtual Max Cut Laboratory. It was used to create static screens representing the Experiment, Connection, Execution, Fallback and Analytic stages. The current prototype contains no implemented code or backend logic. It therefore is a visual representation of the proposed application rather than a functional software system.

Python will be used for the backend implementation in a later development stage. The backend is intended to manage computational processes and support communication with the classical and quantum solver components. Qiskit and Qiskit Aer are intended to support quantum circuit development and local simulation, while IBM Quantum Cloud is intended to provide access to remote quantum hardware. These technologies represent the planned implementation stack and are distinct from the tools already used to create the prototype. At this stage, Figma is the confirmed prototyping tool, while the backend and solver functionality remain to be implemented.

8 SOFTWARE ENGINEERING ASPECTS

8.1 Process & Principles

We decided early on that a strict waterfall approach wouldn't work for us. We're still figuring out the IBM Quantum Runtime side at the same time as we're designing the interface, so we went with an iterative style instead. Every couple of weeks we pick one thin slice of the system (classical solver first, then quantum submission, then the comparison view) and try to get it working end-to-end before moving on. It's not perfectly neat, but it means we can change direction when we discover something new about the cloud API or when classmates give us feedback.

The principles we keep coming back to are pretty basic ones:

- Keep the classical code, the quantum code and the UI as separate as we can, so a change in one place doesn't break everything else.
- Never leave the user staring at a blank screen if IBM is slow or down — always fall back to something useful.
- Make sure we can still point from a requirement we wrote earlier to actual code or a test.
- Prefer simple solutions that a later group of students could still understand over clever ones that only we understand right now.

8.2 Methods & Goals

On the technical side, we're using a mix of things that felt practical. We started with the high-level class diagrams from the previous assignment and refined them as we learned more about how Qiskit Runtime actually behaves. We're prototyping the dual-workbench layout early so we can test whether the "two instruments on a lab bench" idea actually makes sense to other students. For the classical solvers and the graph-to-Hamiltonian conversion, we write unit tests because those parts have to be solid before we even touch the cloud.

The quality goals that matter most to us are:

- The app should still be usable when the IBM backends are queued or temporarily unavailable. That's why we built the simulator path and the lecturer demo mode from the beginning.
- Students should be able to see the actual circuit and the raw counts — hiding the quantum details would defeat the whole point of the project.
- A first-time user should be able to get both a classical and a quantum result in about five minutes.
- The code should still make sense to whoever works on this after us.

8.3 Umbrella Activities

As we transition from our static throwaway prototype to the Python backend implementation, for version control, we will use a normal shared GitHub repository. The main branch will be kept in a working state; we will work on short feature branches, and someone else will review the code before it gets merged. We plan to write commit messages that will still make sense a month later, especially when we are dealing with the weird timeouts that come from the cloud.

Risk management will mostly revolve around the fact that we depend on a service we don't control. The main risk is long queues or sudden unavailability of the IBM backends. We will deal with this by always having the local simulator available, adding a timeout that can fall back to a saved result in demo mode, and logging every job so we can later see how often the real hardware is actually usable. We will also keep a short shared list of other risks (token problems, someone getting sick, scope creep) and glance at it at the start of each cycle.

On the quality side, we will not wait until the end. We will run unit tests whenever we change the classical or conversion code. After each iteration, we will usually grab one or two classmates who haven't seen the latest version and watch them try to use it. We will specifically check whether the messages during a long wait are clear and whether they can finish the basic task without us explaining. Anything that confuses them will go on the list for the next cycle. We will also maintain a simple checklist that links the main requirements to at least one test or demo so we don't quietly forget something important.

Overall, we will try to apply normal software engineering habits, short iterations, clear separation, continuous testing, and honest risk handling—while staying realistic about the fact that this is an undergrad project that depends on a live quantum cloud service.

REFERENCES

[1] J. Ahn, I. DeHaan, E. J. Kim, and E. Lee. 2025. Approximating Maximum Cut on Interval Graphs and Split Graphs beyond Goemans-Williamson. *arXiv preprint arXiv:2507.10436* (2025). doi:10.48550/arxiv.2507.10436

[2] Mina Alipour. 2025. Quantum Software: From Engineering to Interaction. In *2025 IEEE Conference on Cloud and Big Data Computing (CBDCom)*. IEEE, 1–8.

[3] Amazon Web Services. 2024. Amazon Braket Console. AWS Documentation. https://aws.amazon.com/braket/

[4] Graph Online. 2024. Graph Online: Graph Editor and Visualiser. Web application. https://graphonline.ru/

[5] Gian Giacomo Guerreschi and Anne Y. Matsuura. 2019. QAOA for Max-Cut requires hundreds of qubits for quantum speed-up. *Scientific Reports* 9 (2019), 6903. doi:10.1038/s41598-019-43176-9

[6] Nils Herrmann, Daanish Arya, Florian Preis, Stefan Prestel, Marcus W. Doherty, Angus Mingare, and Jason C. Pillay. 2023. Quantum Utility – Definition and Assessment of a Practical Quantum Advantage. *arXiv preprint arXiv:2303.02138* (2023). arXiv:2303.02138 [quant-ph]

[7] IBM Quantum. 2024. IBM Quantum Composer. IBM Quantum Platform. https://quantum.ibm.com/composer

[8] IBM Quantum. 2024. Qiskit Textbook: Quantum Approximate Optimization Algorithm. Qiskit Documentation. https://qiskit.org/learn/

[9] IBM Quantum. 2024. Solving Max-Cut Problems Using QAOA with Qiskit. Qiskit Documentation. https://qiskit.org/documentation/

[10] Open-source contributors. 2024. Max-Cut Visualiser (GitHub repository). GitHub. https://github.com/

[11] Gokul Subramanian Ravi, Kaitlin N. Smith, Pranav Gokhale, and Frederic T. Chong. 2021. Quantum Computing in the Cloud: Analyzing Job and Machine Characteristics. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, Storrs, CT, USA, 39–50.

[12] L. Trevisan. 2009. Max cut and the smallest eigenvalue. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 263–272. doi:10.1145/1536414.1536452

[13] Krishna Upadhyay, Vinaik Chhetri, A. B. Siddique, and Umar Farooq. 2025. Analyzing the Evolution and Maintenance of Quantum Software Repositories. In *2025 IEEE International Conference on Quantum Software (QSW)*. IEEE, Helsinki, Finland, 173–184.

[14] J. Wurtz and D. Lykov. 2021. Fixed-angle conjectures for the quantum approximate optimization algorithm on regular MaxCut graphs. *Physical Review A* 104 (2021). doi:10.1103/physreva.104.052419

[15] Benjamin Zec, Lukas Schmidbauer, Maja Franz, and Wolfgang Mauerer. 2026. Towards Tensor-Network SAT-Solvers for Quantum-Classical Workflows. *arXiv preprint arXiv:2608.02041* (2026). arXiv:2608.02041 [quant-ph]

[16] Leo Zhou, Sheng-Tao Wang, Soonwon Choi, Hannes Pichler, and Mikhail D. Lukin. 2020. Quantum Approximate Optimization Algorithm: Performance, Mechanism, and Implementation on Near-Term Devices. *Physical Review X* 10, 2 (2020), 021067. doi:10.1103/PhysRevX.10.021067